



**UADER – Facultad de Ciencia y Tecnología – Licenciatura en
Sistemas**

Bases de Datos NSQL – Walter Bel

Trabajo Práctico 1 - Introducción Redis

Alumno: Benedetti Valentino

Año lectivo: 2026

1. Compruebe que el servidor esté funcionando, utilizando el comando PING (el servidor devolverá PONG si está funcionando bien).

```
C:\Users\valen>docker exec -it redis-db-2026 redis-cli
127.0.0.1:6379> ping
PONG
```

2. Inserte 5 valores cualquiera a la base de datos 0, utilizando el comando SET clave valor y compruebe que hayan quedado guardados usando el comando GET clave.

```
127.0.0.1:6379> set 1 boca
OK
127.0.0.1:6379> set 2 river
OK
127.0.0.1:6379> set 3 san
OK
127.0.0.1:6379> set 4 racing
OK
127.0.0.1:6379> set 5 independiente
OK
127.0.0.1:6379> get 1
"boca"
```

3. Cambie a la base de datos 1, utilizando el comando SELECT num_db, e ingrese nuevamente 5 valores.

```
127.0.0.1:6379> set 1 juniors
OK
127.0.0.1:6379> set 2 plate
OK
127.0.0.1:6379> set 3 lorenzo
OK
127.0.0.1:6379> set 4 academia
OK
127.0.0.1:6379> set 5 rojo
OK
127.0.0.1:6379> get 1
"juniors"
127.0.0.1:6379> get 2
"plate"
127.0.0.1:6379> get 3
"lorenzo"
127.0.0.1:6379> get 4
"academia"
127.0.0.1:6379> get 5
"rojo"
```

4. Determine la cantidad de elementos almacenados en la base de datos (1), utilizando el comando DBSIZE.

```
127.0.0.1:6379> dbsize  
(integer) 5
```

5. Elimine todos los valores de la base de datos (1), utilizando el comando FLUSHDB, verifique si la operación fue exitosa.

```
127.0.0.1:6379> flushdb  
OK  
127.0.0.1:6379> dbsize  
(integer) 0
```

6. Liste todas las claves de la base de datos (0), utilizando el comando KEYS *.

```
127.0.0.1:6379> keys *  
1) "3"  
2) "1"  
3) "5"  
4) "2"  
5) "4"  
c
```

7. Determine si la clave "xyz" existe en la base de datos (0), utilizando el comando EXISTS clave.

```
127.0.0.1:6379> exists xyz  
(integer) 0
```

8. Cambiar el valor de una clave existente en la base de datos (0) y retornar el valor anterior, utilizando el comando GETSET clave nuevo_valor, verifique si la operación fue exitosa.

```
127.0.0.1:6379> getset 1 boca  
"boca"  
127.0.0.1:6379> get 1  
"bocaaaa"
```

9. Eliminar una clave de la base de datos (0), utilizando el comando DEL clave, verifique si la operación fue exitosa.

```
127.0.0.1:6379> del 2  
(integer) 1  
127.0.0.1:6379> get 2  
(nil)
```

10. Mover una clave y valor de una base de datos a otra (de la 0 a la 1), utilizando el comando MOVE clave num_db, verifique si la operación fue exitosa.

```
127.0.0.1:6379> move 3 5
(integer) 1
127.0.0.1:6379> select 5
OK
127.0.0.1:6379[5]> get 3
"san"
```

11. Seleccione una clave de manera aleatoria, utilizando el comando RANDOMKEY, pruébelo más de una vez para ver el resultado.

```
127.0.0.1:6379> randomkey
"4"
127.0.0.1:6379> randomkey
"5"
127.0.0.1:6379> randomkey
"1"
```

12. Determine el tipo de dato del valor asociada a una clave, utilizando el comando TYPE clave.

```
127.0.0.1:6379> type 4
string
```

13. Cree una lista de personajes de Star Wars que contenga al menos cinco nombres en la base de datos (0), utilizando el comando LPUSH clave valor1 valor2 valor3.

```
127.0.0.1:6379> LPUSH starwars "luke" "leia" "han" "vader" "chewie"
(integer) 5
```

14. Listar el contenido completo de la lista, utilizando el comando LRANGE clave inicio fin (-1 representa el último elemento de la lista).

```
127.0.0.1:6379> LRANGE starwars 0 -1
1) "chewie"
2) "vader"
3) "han"
4) "leia"
5) "luke"
```

15. Obtenga el valor almacenado en el índice 0 de la lista creada en el punto anterior, utilizando el comando LINDEX clave 0. ¿Es el primero que se cargó?

```
127.0.0.1:6379> LINDEX starwars 1  
"chewie"
```

16. Agregue dos nombres más a la lista creada en el punto 13, utilizando LPUSH, verifique si la operación fue exitosa.

```
127.0.0.1:6379> LPUSH starwars "yoda" "obi-wan"  
(integer) 7  
127.0.0.1:6379> LRANGE starwars 0 -1  
1) "obi-wan"  
2) "yoda"  
3) "chewie"  
4) "vader"  
5) "han"  
6) "leia"  
7) "luke"
```

17. Determine cuantos elementos tiene la lista, utilizando el comando LLEN clave.

```
127.0.0.1:6379> LLEN starwars  
(integer) 7
```

18. Elimine el primer elemento de la lista, utilizando el comando LPOP, verifique si la operación fue exitosa.

```
127.0.0.1:6379> LPOP starwars  
"obi-wan"  
127.0.0.1:6379> LRANGE starwars 0 -1  
1) "yoda"  
2) "chewie"  
3) "vader"  
4) "han"  
5) "leia"  
6) "luke"
```

19. Ingrese un valor a la lista antes y después de otro valor de la lista, utilizando el comando LINSERT clave BEFORE|AFTER valor nuevo_valor, verifique si las operaciones fueron exitosas.

```

127.0.0.1:6379> LINSERT starwars BEFORE "vader" "R2D2"
(integer) 7
127.0.0.1:6379> LINSERT starwars AFTER "vader" "C3PO"
(integer) 8
127.0.0.1:6379> LRANGE starwars 0 -1
1) "yoda"
2) "chewie"
3) "R2D2"
4) "vader"
5) "C3PO"
6) "han"
7) "leia"
8) "luke"

```

20. Ingrese en una lista un mismo valor en distintas posiciones, elimine la primera ocurrencia del mismo, utilizando el comando LREM clave cantidad valor, verifique si la operación fue exitosa.

```

127.0.0.1:6379> LPUSH starwars "luke"
(integer) 9
127.0.0.1:6379> LRANGE starwars 0 -1
1) "luke"
2) "yoda"
3) "chewie"
4) "R2D2"
5) "vader"
6) "C3PO"
7) "han"
8) "leia"
9) "luke"
127.0.0.1:6379> LREM starwars 1 "luke"
(integer) 1
127.0.0.1:6379> LRANGE starwars 0 -1
1) "yoda"
2) "chewie"
3) "R2D2"
4) "vader"
5) "C3PO"
6) "han"
7) "leia"
8) "luke"

```

21. Elimine todas las ocurrencias del valor repetido con una sola sentencia, verifique si la operación fue exitosa.

```

127.0.0.1:6379> LREM starwars 0 "luke"
(integer) 1

```

22. Elimine el último valor de una lista, utilizando el comando RPOP clave, verifique si la operación fue exitosa.

```

127.0.0.1:6379> RPOP starwars
"leia"

```

23. Cambiar el nombre de una clave, utilizando el comando RENAME clave nueva_clave, verifique si la operación fue exitosa.

```
127.0.0.1:6379> RENAME starwars personajes_sw  
OK
```

24. Fuerce a Redis a pasar los datos de memoria a disco, utilizando el comando SAVE.

```
127.0.0.1:6379> SAVE  
OK
```

25. Ordene la lista de elementos, utilizando el comando SORT clave (numérico) o SORT clave (alfanumérico).

```
127.0.0.1:6379> SORT personajes_sw ALPHA  
1) "C3PO"  
2) "R2D2"  
3) "chewie"  
4) "han"  
5) "vader"  
6) "yoda"
```

26. Ordene ahora la lista de manera descendente.

```
127.0.0.1:6379> SORT personajes_sw ALPHA DESC  
1) "yoda"  
2) "vader"  
3) "han"  
4) "chewie"  
5) "R2D2"  
6) "C3PO"
```

27. Cambie el valor de uno de los elementos de la lista utilizando el comando LSET, verifique si la operación fue exitosa.

```
127.0.0.1:6379> LSET personajes_sw 0 "anakin"  
OK
```

28. Ingrese una clave-valor solo si la misma no existe con un solo comando.

```
127.0.0.1:6379> SETNX profe "juan"  
(integer) 1
```

29. Ingrese una clave-valor de manera temporal con el comando SETEX, verifique si la

operación fue exitosa, es decir si se ingresó y luego del tiempo expiro.

```
127.0.0.1:6379> SETEX temporal 15 "bomba"  
OK
```

30. Determine el tiempo restante para que expire una clave usando el comando PTTL.

```
127.0.0.1:6379> PTTL temporal  
(integer) -2
```

31. Cree un conjunto geoespacial cerveceros con el comando GEOADD, de las cervecerías artesanales de la ciudad (Lagash, Drakkar, Biguá, Tractor, 7 Colinas, Ambar).

```
127.0.0.1:6379> GEOADD cerveceros -58.2310 -32.4810 "Lagash" -58.2320 -32.4820 "Drakkar" -58.2330 -32.4830 "Bigua" -58.2340 -32.4840 "Tractor" -58.2350 -32.4850 "7 Colinas" -58.2360 -32.4860 "Ambar"  
(integer) 6
```

32. Consulte las coordenadas de Lagash con el comando GEOPOS.

```
127.0.0.1:6379> GEOPOS cerveceros "Lagash"  
1) 1) "-58.231001794338226"  
2) "-32.481000529064936"
```

33. Agregue la FCyT al conjunto de cerveceros

```
127.0.0.1:6379> GEOADD cerveceros -58.2290 -32.4790 "FCyT"  
(integer) 1
```

34. Determine la distancia en metros y kilómetros desde la FCyT hasta Lagash y Drakkar, utilizando GEODIST.

```
127.0.0.1:6379> GEODIST cerveceros "FCyT" "Lagash" m  
"291.0800"  
127.0.0.1:6379> GEODIST cerveceros "FCyT" "Drakkar" km  
"0.4363"
```

35. Determine las cervecerías que se encuentran dentro del radio de 1km utilizando el comando GEORADIUS.


```
127.0.0.1:6379> GEORADIUS cerveceros -58.2290 -32.4790 1 km
1) "7 Colinas"
2) "Tractor"
3) "Bigua"
4) "Drakkar"
5) "Lagash"
6) "FCyT"
```

36. Subsribase a los siguientes canales: FCyT, IDTILAB, NSQL utilizando el SUBSCRIBE, para implementar el paradigma de mensajería Publish/Subscribe.

(ABAJO ESTÁ LA TERMINAL) -----

37. Ahora desde otro terminal de redis publique el mensaje “Hola a Mundo” en el canal NSQL con el comando PUBLISH.

(ABAJO ESTÁ LA TERMINAL) -----

38. Desde otro terminal suscribase a los canales IDTILAB, “STAR WARS”

(ABAJO ESTÁ LA TERMINAL) -----

39. Ahora desde otro terminal de redis publique el mensaje “Hola a todos los miembros del laboratorio” en el canal NSQL con el comando PUBLISH. 40. Chequear los canales activos con el comando PUBSUB CHANNELS, abra otro terminal y suscribase al canal “STAR WARS” y verifique nuevamente los canales activos.

(ABAJO ESTÁ LA TERMINAL) -----

41. Ahora cheque el número de suscriptores de cada canal activo con el comando PUBSUB NUMSUB.

terminal 1

```
127.0.0.1:6379> SUBSCRIBE FCyT IDTILAB NSQL
1) "subscribe"
2) "FCyT"
3) (integer) 1
1) "subscribe"
2) "IDTILAB"
3) (integer) 2
1) "subscribe"
2) "NSQL"
3) (integer) 3
1) "message"
2) "NSQL"
3) "Hola a Mundo"
1) "message"
2) "NSQL"
3) "Hola a todos los miembros del laboratorio"
```

terminal 2

```
C:\Users\valen>docker exec -it redis-db-2026 redis-cli
127.0.0.1:6379> PUBLISH NSQL "Hola a Mundo"
(integer) 1
127.0.0.1:6379> PUBLISH NSQL "Hola a todos los miembros del laboratorio"
(integer) 1
127.0.0.1:6379> PUBSUB CHANNELS
1) "STAR WARS"
2) "NSQL"
3) "IDTILAB"
4) "FCyT"
127.0.0.1:6379> PUBSUB NUMSUB FCyT IDTILAB NSQL "STAR WARS"
1) "FCyT"
2) (integer) 1
3) "IDTILAB"
4) (integer) 2
5) "NSQL"
6) (integer) 1
7) "STAR WARS"
8) (integer) 1
```

terminal 3

```
C:\Users\valen>docker exec -it redis-db-2026 redis-cli
127.0.0.1:6379> SUBSCRIBE IDTILAB "STAR WARS"
1) "subscribe"
2) "IDTILAB"
3) (integer) 1
1) "subscribe"
2) "STAR WARS"
3) (integer) 2
```

42. El comando UNSUBSCRIBE nos permite cancelar la subscripción a un canal o a todos, pero no se puede ejecutar desde el redis-cli, se utiliza cuando estamos trabajando en una aplicación.